

# Context Engineering Learning Notes

Source 1  
Anthropic Engineering Blog

Source 2  
PromptingGuide.ai

Source 3  
LangChain Blog

## THE CORE DEFINITION

**Context Engineering** is the art and science of filling the LLM context window with just the right information — at the right time — to maximize the likelihood of a desired outcome. It is the natural evolution of prompt engineering, extending beyond single prompts to managing the full lifecycle of information an agent sees across every inference step.

## KEY VOCABULARY

|                                                                                               |                                                                                               |
|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Context Window</b><br>The finite set of tokens the LLM can see at one time — its 'RAM'     | <b>Context Rot</b><br>Performance degrades as context length grows; attention budget depletes |
| <b>Compaction</b><br>Summarising a full conversation to restart with a fresh, smaller context | <b>Scratchpad</b><br>Agent writes notes outside context window; reads them back later         |
| <b>RAG</b><br>Retrieval-Augmented Generation — fetch relevant knowledge just-in-time          | <b>Sub-agents</b><br>Specialised agents with isolated context windows handling sub-tasks      |

## CORE ARGUMENT

Prompt engineering focused on writing good instructions. Context engineering asks a bigger question: what is the optimal configuration of ALL tokens the model sees — system prompt, tools, examples, message history, retrieved data — to produce the desired behaviour at every inference step?

## PROMPT ENGINEERING vs CONTEXT ENGINEERING

| Prompt Engineering           | Context Engineering                    |
|------------------------------|----------------------------------------|
| One-shot or few-shot tasks   | Multi-turn agents in a loop            |
| Focus: wording of the prompt | Focus: entire token state at inference |
| Static — written once        | Iterative — curated every turn         |
| Single system prompt         | System + tools + memory + history      |

## WHY CONTEXT SIZE HURTS PERFORMANCE

## Context Rot

- As token count rises, recall accuracy falls — attention budget gets spread thin
- Transformer architecture: every token attends to every other →  $n^2$  relationships
- Models trained on shorter sequences; longer context = less-trained territory
- Performance degrades gradually, not off a cliff — but planning around it is essential

## THE GOLDEN RULE

*"Find the **smallest** possible set of **high-signal tokens** that maximise the likelihood of your desired outcome."*

## 5 COMPONENTS OF EFFECTIVE CONTEXT

## System Prompts

- Aim for the 'right altitude': not too prescriptive, not too vague
- Avoid hardcoded if-else logic — it creates fragility
- Avoid false assumptions of shared context — spell things out
- Use XML tags or Markdown headers to organise sections clearly
- Start minimal, then add instructions based on observed failures

## Tools

- Define tools so a human can unambiguously pick the right one

|                                                            |
|------------------------------------------------------------|
| • Avoid bloated tool sets — overlap causes model confusion |
| • Tools should be self-contained and robust to error       |
| • Input parameters must be descriptive and unambiguous     |
| • Token-efficient tool outputs help keep context lean      |

| Few-shot Examples                                                       |
|-------------------------------------------------------------------------|
| • Curate diverse, canonical examples — not an exhaustive edge-case list |
| • Think of examples as 'pictures worth a thousand words' for the LLM    |
| • Quality over quantity — 3 great examples beat 20 mediocre ones        |

| Just-in-time Retrieval                                                 |
|------------------------------------------------------------------------|
| • Agents hold lightweight identifiers (file paths, URLs, queries)      |
| • Load data into context dynamically at runtime — not all upfront      |
| • Mirrors human cognition: we index, not memorise, large corpora       |
| • Enables progressive disclosure — each tool call reveals next clue    |
| • Trade-off: runtime exploration is slower than pre-computed retrieval |

LONG-HORIZON STRATEGIES

| Compaction                                                                                                                                  | Structured Note-taking                                                                                                                                | Sub-agent Architecture                                                                                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Summarise full context window when near limit. Preserve key decisions, discard raw tool outputs. Claude Code auto-compacts at 95% capacity. | Agent writes persistent notes (NOTES.md, to-do list) outside the context window. Notes get re-injected at future turns. Enables cross-session memory. | One lead agent delegates to specialised sub-agents, each with a clean context. Sub-agents return 1–2k token summaries, not raw output. Parallel exploration. |

#### THE BROADER DEFINITION

Context engineering is the process of designing and optimising instructions and relevant context for LLMs and advanced AI models to perform their tasks effectively. It encompasses text-based and multimodal models, extending across the full prompt engineering toolkit.

#### WHAT CONTEXT ENGINEERING INCLUDES

- Designing and managing prompt chains
- Tuning instructions and system prompts
- Managing dynamic elements (user input, date/time, runtime state)
- Searching and preparing relevant knowledge (RAG)
- Query augmentation and reformulation
- Tool definitions and instructions for agentic systems
- Preparing and optimising few-shot demonstrations
- Structuring inputs and outputs (delimiters, JSON schema)
- Short-term memory: managing conversational state and history
- Long-term memory: retrieving relevant knowledge from vector stores
- Filtering out noisy, low-signal information
- Setting up eval pipelines to measure whether tactics work

#### PRACTICAL WALKTHROUGH — BUILDING A PLANNING AGENT

The guide walks through a real multi-agent deep research system built in n8n. The Search Planner sub-agent illustrates six concrete context engineering decisions:

##### 1. Instructions

High-level role + task description. Example: 'You are an expert research planner. Break down a complex query into specific search subtasks.' This alone is insufficient — it is just the starting point.

##### 2. User Input with delimiters

Wrap user input in XML tags: What's the latest from OpenAI?. Delimiters prevent confusion between what comes in and what should come out.

##### 3. Structured output spec

Explicitly define field names, types, and ranges. Example: priority: int (1=highest to 5=lowest). Without this, the model may choose a 1–10 scale or invent fields.

##### 4. Few-shot JSON example

Provide a concrete JSON output example. Tools like n8n auto-generate a schema from it, forcing the model to produce consistently parseable structured output.

## 5. Dynamic context injection

Inject the current date/time via a function call: `{{ $now.toISO() }}`. Without this, the model guesses dates and produces inaccurate search queries.

## 6. Eval pipeline

Context engineering is iterative. Measure whether each change actually improves output quality. Formal evals are not optional — they are part of the discipline.

### KEY INSIGHT FROM THIS SOURCE

*Context engineering is not just for agents. It applies to every LLM system — chatbots, classifiers, generators. The difference is that agents make the stakes higher because errors in context compound across dozens of autonomous tool calls.*

## THE FRAMING

LangChain frames the LLM as a CPU and the context window as RAM. Just as an OS decides what data lives in RAM at any moment, context engineering decides what information the LLM sees at each inference step of an agent's trajectory.

## 4 WAYS CONTEXT CAN FAIL

|                            |                                                                              |
|----------------------------|------------------------------------------------------------------------------|
| <b>Context Poisoning</b>   | A hallucination gets written into context and corrupts all future reasoning. |
| <b>Context Distraction</b> | Critical information gets buried under noise and the model ignores it.       |
| <b>Context Confusion</b>   | Superfluous, irrelevant context distracts the model from its actual task.    |
| <b>Context Clash</b>       | Contradictory information in context leads to inconsistent model behaviour.  |

## THE FOUR STRATEGIES (LANGCHAIN'S FRAMEWORK)

## WRITE

|                    |                                                                                                                                                                         |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Scratchpads</b> | Agent writes notes to a file or state object during a task. Information persists outside the context window and is read back when needed.                               |
| <b>Memories</b>    | Auto-generate long-term memories from past interactions. Reflexion introduced post-turn reflection. Products like Cursor, ChatGPT, and Windsurf use this in production. |

## SELECT

|                        |                                                                                                                                                  |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>From scratchpad</b> | Read back stored notes via tool call or by exposing the relevant part of agent runtime state to the LLM.                                         |
| <b>Memories</b>        | Use embeddings or knowledge graphs to retrieve only the most relevant memories — not all of them. Poor selection = unexpected context injection. |
| <b>Tools via RAG</b>   | Apply RAG to tool descriptions so the agent only sees the tools relevant to the current step. Improves tool selection accuracy by 3×.            |
| <b>Knowledge (RAG)</b> | For code agents: combine embedding search, AST chunking, grep/file search, knowledge graphs, and re-ranking for reliable retrieval.              |

## COMPRESS

|                      |                                                                                                                                 |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------|
| <b>Summarisation</b> | Distil agent trajectory using an LLM. Apply recursively or hierarchically. Claude Code auto-compacts at 95% context usage.      |
| <b>Trimming</b>      | Remove older messages using hard-coded heuristics. Faster than summarisation; useful for predictable context overflow patterns. |

## ISOLATE

|                      |                                                                                                                                           |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Multi-agent</b>   | Split tasks across sub-agents, each with its own clean context window. Parallel exploration.<br>Trade-off: up to 15× more tokens overall. |
| <b>Code sandbox</b>  | HuggingFace CodeAgent pattern: LLM outputs code, code runs in a sandbox, only selected return values go back into the LLM context.        |
| <b>Runtime state</b> | Store context in a typed state object. Developers choose exactly which fields are exposed to the LLM at each step.                        |

#### KEY INSIGHT FROM LANGCHAIN'S CEO

*"Context engineering is such a good term. It actually really describes everything we've done at LangChain without knowing that term existed. Traces tell you what's in your context — and that's so important. The source of truth for agents is a combination of code and traces."*

# Synthesis — What All 3 Sources Agree On

## Context is finite and precious

All three sources use the RAM/attention budget metaphor. Treating context as unlimited is the most common mistake engineers make.

## Smallest is best

The optimal context is not the most complete — it is the most relevant. More tokens = more diluted attention. Curate aggressively.

## Context engineering is iterative

You cannot write it once and be done. Each agent turn may require a different configuration. Build eval pipelines to measure quality continuously.

## Tools are part of context

Tool descriptions, schemas, and results consume token budget. Poorly designed tools bloat context and create confusion. Design tools like you design functions.

## Memory is external storage

Scratchpads, note files, and vector stores are all forms of external memory. Agents that write and read their own notes vastly outperform those that keep everything in-window.

## Sub-agents are a context strategy

Not just an architecture choice. Each sub-agent gets a fresh, focused context. The lead agent receives a compact summary — not raw output. This is context compression at the system level.

### QUICK DECISION GUIDE — WHICH STRATEGY TO USE?

| If you need to...               | Use this strategy        | Example tool/pattern       |
|---------------------------------|--------------------------|----------------------------|
| Persist info during a task      | Write — Scratchpad       | NOTES.md, state object     |
| Remember things across sessions | Write — Long-term memory | Vector store, file system  |
| Load only what's needed         | Select — RAG             | Embedding search + rerank  |
| Reduce too many tools           | Select — Tool RAG        | RAG over tool descriptions |
| Handle context overflow         | Compress — Summarise     | LLM-based compaction       |
| Remove old irrelevant turns     | Compress — Trim          | Message trimming heuristic |
| Run parallel sub-tasks          | Isolate — Multi-agent    | LangGraph sub-agents       |
| Keep heavy objects out of LLM   | Isolate — Sandbox        | CodeAgent + E2B sandbox    |

## SELF-TEST QUESTIONS

Q1. What is the difference between prompt engineering and context engineering?

Q2. Why does a longer context window not solve the performance problem?

Q3. Name 3 things that count as 'context' beyond the system prompt.

Q4. What is context rot and what causes it architecturally?

Q5. When should you use compaction vs sub-agents for long-horizon tasks?

Q6. What are the 4 context failure modes from LangChain?

Q7. Describe the 'just-in-time' retrieval pattern and its trade-offs.

Q8. How does structured output specification help context engineering?

Q9. What is the difference between episodic, procedural, and semantic memory?

Q10. Why is tool design a context engineering problem?